Feature: A client's site can book once its business has a Website

From build-plan: feature 25b

Size: light - one piece of logic (which site may call which business's booking routes); no data change, no screen. **Status:** verified **Branch:** feature/25b-a-client-s-site-can-book-once-its-business-has-a-website

Goal

A new client's site books as soon as the agency admin fills in the business's Website, with no Railway edit and no redeploy. The API lets a browser on that business's own website call that business's public booking routes; every other site stays refused, as today.

In scope

- **The rule**, in `publicCorsMiddleware`: a browser's Origin is allowed when 1. it is on the fixed list, as today: `WIDGET_ORIGINS` (now only the laptop's dev sites) and the dashboard (`APP_ORIGIN`); or 2. the path names a business (`/public/<slug>/...`) and the Origin is that business's own Website: `https://<domain>` or `https://www.<domain>`, the domain read from its Website field the same way 25a does (`website-domain.ts`). Anything else gets no `Access-Control-Allow-Origin`, so the browser blocks it, exactly as an unlisted site does today.
- **Only that business.** Business A's website never opens business B's routes: the Website compared is the one of the business in the path.
- **Not the booking page.** `/public/bookings/<token>...` (the customer's own booking page, served by the dashboard) keeps the fixed list only.
- **https only.** An `http://` Origin never matches a Website (the Website field already requires `https://`).
- **Fast.** A business's Website is looked up by slug and kept for 60 seconds inside the process, so a busy site does not query the database on every call and a changed Website works within a minute. An unknown slug, or a lookup that fails, allows nothing beyond the fixed list.
- **Docs.** `AGENTS.md` and the `.env.example` comment say `WIDGET_ORIGINS` is for the laptop's dev sites; live client sites come from each business's Website.

Out of scope

- Removing Face and Body's and IA a la Orden's entries from Railway's `WIDGET_ORIGINS` (harmless; tidied later, by hand, on Frank's yes).

- Several websites per business, or subdomains other than `www`.
- Changing the Website field itself or who may edit it (still the agency admin).

Build loop

After Frank's yes on this plan, the one step runs straight through: build, tests, the build log, commit and push to the feature branch. The review comes once, at `/complete` (light). Stops only for the three reasons in `AGENTS.md`.

Build steps

- [ ] 1. The origin rule in `publicCorsMiddleware` with a small cached lookup of a business's Website by slug, its tests, and the docs. **Done when:** backend tests pass, including new ones for: the business's own Website allowed (with and without `www`, on a read and on the preflight for a booking POST); another business's Website refused on this business's routes; an `http://` Origin refused; the booking page routes not opened by a Website; an unknown slug refused; the fixed list still allowed. Backend build passes.

Files / areas

- `backend/middleware/public-middleware/public-cors-middleware.ts`
- a new `backend/lib/...` file: the cached Website-by-slug lookup (one export)
- `backend/lib/email/website-domain.ts` (reused, not changed)
- `backend/routes/public-booking-links-routes.test.ts` (its CORS tests), or a new middleware test
- `AGENTS.md`, `.env.example` (Frank's uncommitted copy)

Data / contracts

- No migration. `organization.website` (https URL, set by the agency admin) decides; `organization.slug` finds it.
- Response headers unchanged in shape: `Access-Control-Allow-Origin` names the calling Origin when allowed, never `*`, never with credentials.

Testing

Backend Vitest against the local database (the route tests' existing setup), each test with its own throwaway business and Website, so the 60-second cache never carries one test into another.

Notes for the AI

- One export per backend file; types end in `Type`.
- The lookup never logs an Origin or a Website.

Implementation walkthrough

The rule (backend/middleware/public-middleware/public-cors-middleware.ts). The public routes' CORS check now takes a function. An Origin on the fixed list (the dashboard, and `WIDGET_ORIGINS`, now only the laptop's dev sites) is allowed as before. Any other Origin must be https; then the business named in the path (`/public/<slug>/...`, never the customer's own booking page under `/public/bookings/`) is looked up, and the Origin is allowed only when it is exactly `https://<its domain>` or `https://www.<its domain>`. Everything else gets no `Access-Control-Allow-Origin`, so the browser blocks it. A failed lookup logs one line and allows nothing beyond the fixed list.

The lookup (backend/lib/booking/website-domain-for-slug.ts). A business's Website by its address, read with 25a's `website-domain.ts`, refused before any query when the address has the wrong shape, and kept for a minute so busy sites do not query the database on every call. The first review's F-544 capped this memory at 1000 entries and kept non-https Origins away from it, so made-up addresses cannot grow it without limit.

Tests. Five route tests (`public-booking-links-routes.test.ts`): its own Website with and without www, on a read and on a booking's preflight; another business's Website refused; http, a look-alike, another subdomain and an unknown business refused; the booking page not opened by a Website; the dashboard still allowed. Two lookup tests (`website-domain-for-slug.test.ts`): kept for a minute then read again; an unknown business or wrong-shaped address has none.

Docs. AGENTS.md has a "Client sites (25b)" paragraph; the `.env.example` comments were updated in Frank's own uncommitted copy (F-545).

Carried. F-547 (P2): a request with any https Origin and a new address still costs one indexed database query, outside the visitor limit, and enough of them empty the minute's memory; loading every business's address in one query a minute would remove it. F-545 (P3, docs), F-546 (P3, the failed-lookup rule has no test).

Findings

25b/F-544 [P1] closed - The Website cache grows without limit, and a preflight fills it with no rate limit

File: backend/lib/booking/website-domain-for-slug.ts:15,28; backend/middleware/public-middleware/public-cors-middleware.ts:29-34; backend/app.ts:54 **Found:** 2026-10-27 by /audit independent current (scope: aa8627d..258544c; lenses: quality, security, performance, tests) **Why it**

matters: `kept` is a module-level Map keyed by the slug taken from the request path, and nothing ever removes an entry: an expired entry is only replaced if the same slug is asked for again. Any slug-shaped segment (`/^[a-z0-9][a-z0-9-]{0,47}$/` , an effectively endless space) adds one entry and one database query, real business or not. The CORS origin function runs for every `/public/<x>/...` request, including requests with no `Origin` header (`origin` is `""` , which can never match a Website) and every `OPTIONS` preflight. `publicCorsMiddleware` is mounted before `publicRateLimitMiddleware` (app.ts:54), and Hono's cors answers `OPTIONS` itself without calling `next()` , so a stranger sending preflights to `/public/<random>/x` gets one uncached query and one permanent Map entry per request with no rate limit at all; GETs also pay the lookup before the limiter counts them. Before this delta a preflight was an array lookup. Left alone, the API process (which also runs the job worker) grows until it is killed, and the database takes an unthrottled query stream. This defeats the spec's "Fast" rule, which exists so calls do not hit the database. **Suggested fix:** Skip the lookup unless the Origin starts with `https://` (an empty or `http://` Origin can never match). Bound the cache: for example sweep expired entries when the Map passes a few thousand, or cap it and drop the oldest. Optionally count preflights against the visitor's read limit, or run the limiter before the lookup. Add a test that many unknown slugs leave the cache bounded. **Resolution:** fixed 2026-10-27 on feature/25b: an Origin that is not https never reaches the lookup (public-cors-middleware.ts), and the kept map is capped at 1000 entries, starting over past it (website-domain-for-slug.ts), so memory stays bounded. A valid-shaped slug from an https Origin still costs one indexed query per minute per slug; preflights stay outside the visitor limit, as before 25b (moving the limiter in front of CORS would strip the CORS headers from its own 429 answers). Closed 2026-10-27 by /audit independent current (fresh subagent, aa8627d..f793bc1): re-examined public-cors-middleware.ts:29-42 and website-domain-for-slug.ts:15-32. An empty, `null` or `http://` Origin returns before `businessSlugOf` and the lookup; a junk segment is refused by `organizationSlugValidationSchema` before any query; the map holds at most 1000 entries and is cleared before the 1001st, so the process memory growth (the P1 harm) is gone, and the repair adds no new defect in memory or correctness. The remaining part, one uncached query per novel slug from an https Origin with no limit in front of it, is a smaller, separate risk and is recorded as F-547 (P2).

Independent review

Status: passed **Target commit:** f793bc15ccdb437e37f34934cda5eb76a89ab118 **Base commit:** aa8627dc45f0c3f93ccb24187403abaa3f2f839c **Base ref:** main **Spec hash:** a10376e6d133f590b47750c12077564a15fea6de9815c01dde0f925a8b98c3f4 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-27T16:51:00Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-27T16:54:44Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Commands

- `git rev-parse HEAD`, `git merge-base main HEAD`, `sha256sum blueprint/context/current-feature.md`, `git status --porcelain`: pass (target, base and spec hash match; only review.md differed)
- `npm run build --workspace=backend`: pass
- `npm run test --workspace=backend`: pass (97 files, 1105 tests)
- `npx vitest run lib/booking/website-domain-for-slug.test.ts routes/public-booking-links-routes.test.ts --reporter=verbose` (in backend): pass, the seven 25b tests ran and passed

Evidence

- Origin rule (public-cors-middleware.ts:29-42): fixed list first; non-https Origin (including empty and `null`) returns before any lookup; exact match on `https://<domain>` or `https://www.<domain>` only, so look-alike suffixes and other subdomains are refused; no credentials header.
- Path parsing: `c.req.path` is decodeURI'd by Hono 4.13.14, the same path the router uses; `bookings` is excluded so `/public/bookings/<token>...` keeps the fixed list; a segment with `%` or upper case fails `organizationSlugValidationSchema` before any query, so the CORS slug can never name a different business than the route's `:slug`.
- Tenant isolation: the Website compared is the one of the slug in the path; the booking POST resolves its business from the same `:slug` (public-bookings-routes.ts:57-62). Test "another business's Website is refused on this business's routes" passes.
- Cache: bounded at 1000 entries (website-domain-for-slug.ts:30); the 60-second keep is proved at 59 s and 61 s through the `now` seam.
- Failure: a thrown lookup logs only `safeErrorReason` (Postgres code, never the query or values) and returns no Allow-Origin; read, not tested (F-546).
- Hono cors appends `Vary: Origin` on both preflight and normal answers, so a shared cache cannot replay one site's Allow-Origin to another.

Findings

- F-544 [P1] closed: the cache is bounded and non-https Origins never reach the lookup.
- F-545 [P3] open: `.env.example` comments still describe WIDGET_ORIGINS as the live client list.
- F-546 [P3] open (reopened, narrowed): the cache expiry is now tested; the failed-lookup rule still has no test.
- F-547 [P2] open (new): an https-Origin preflight to a novel slug costs one unlimited database query, and 1000 of them clear real businesses' entries.

Remaining risk

- The database-failure path is verified by reading only; no test simulates a failed lookup (F-546).
- Load from F-547 is reasoned from the code path, not measured.

- Check was not required and not run; no browser exercised a real cross-origin call from a client site.